

گزارش پروژه سوم درس نرم افزار ریاضی

بخش اول: درونیابی لاگرانژ و اسپلین مکعبی

نگارش: آرمان صالحی

تاریخ: ۶ تیر ۱۴۰۵

۱ صورت مسئله و داده های ورودی

در این بخش، هدف درونیابی ۶ نقطه داده شده در صفحه و استخراج ضابطه دقیق ریاضی آنها با استفاده از دو روش چندجمله ای لاگرانژ و اسپلین مکعبی است. مختصات نقاط در جدول ۱ آورده شده است.

جدول ۱: نقاط داده شده جهت درونیابی

۶	۵	۴	۳	۲	۱	x
۲	۵	۸	۵	۳	۱	y

۲ محاسبات و ضوابط ریاضی استخراج شده

۱.۲ الف) چندجمله ای لاگرانژ

با توجه به وجود ۶ نقطه متمایز، چندجمله ای گذرنده از این نقاط یک ضابطه واحد از درجه ۵ خواهد بود:

$$P(x) = 0.175x^5 - 2.958x^4 + 18.38x^3 - 52.04x^2 + 68.45x - 31$$

۲.۲ ب) درونیابی اسپلین مکعبی

در این روش، بین هر دو نقطه متوالی یک چندجمله ای درجه ۳ به صورت محلی تعریف می شود. ضوابط ۵ بازه شکل گرفته به صورت منظم در زیر محاسبه شده اند:

$$S_1(x) = +0.6667(x-1)^3 - 2.0000(x-1)^2 + 3.3333(x-1) + 1.0000 \quad (1 \leq x \leq 2)$$

$$S_2(x) = +0.6667(x-2)^3 + 0.0000(x-2)^2 + 1.3333(x-2) + 3.0000 \quad (2 \leq x \leq 3)$$

$$S_3(x) = -2.3333(x-3)^3 + 2.0000(x-3)^2 + 3.3333(x-3) + 5.0000 \quad (3 \leq x \leq 4)$$

$$S_4(x) = +1.6667(x-4)^3 - 5.0000(x-4)^2 + 0.3333(x-4) + 8.0000 \quad (4 \leq x \leq 5)$$

$$S_5(x) = +1.6667(x-5)^3 + 0.0000(x-5)^2 - 4.6667(x-5) + 5.0000 \quad (5 \leq x \leq 6)$$

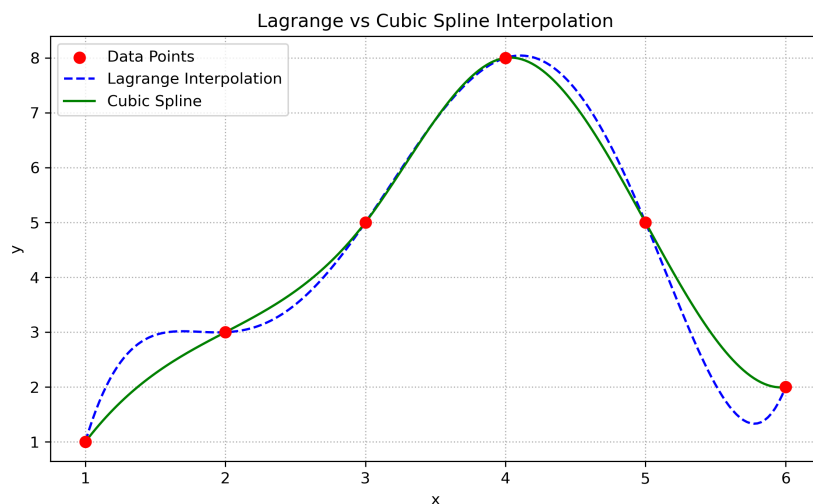
۳ کد پیاده‌سازی پایتون

کد زیر توسعه‌یافته در فایل part1.py، فرآیند محاسبه ضرایب و تولید خروجی بصری را انجام می‌دهد:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import lagrange, CubicSpline
4
5 # 1. Define the given data points
6 x = np.array([1, 2, 3, 4, 5, 6])
7 y = np.array([1, 3, 5, 8, 5, 2])
8
9 # Part A: Lagrange Interpolation
10 poly_lagrange = lagrange(x, y)
11
12 # Part B: Cubic Spline Interpolation
13 cs = CubicSpline(x, y)
14
15 # Part C: Plotting the Interpolations
16 x_dense = np.linspace(1, 6, 200)
17
18 plt.figure(figsize=(9, 5), dpi=120)
19 plt.scatter(x, y, color='red', s=50, zorder=5, label='Data
Points')
20 plt.plot(x_dense, poly_lagrange(x_dense), 'b--', label='
Lagrange Interpolation')
21 plt.plot(x_dense, cs(x_dense), 'g-', label='Cubic Spline')
22
23 plt.title('Lagrange vs Cubic Spline Interpolation')
24 plt.xlabel('x')
25 plt.ylabel('y')
26 plt.legend()
27 plt.grid(True, linestyle=':')
28 plt.savefig('part1_interpolation.png', dpi=300)
29 plt.show()
30
```

۴ خروجی بصری و تحلیل مهندسی

نمودار خروجی حاصل از اجرای کد، در شکل ۱ آورده شده است.



شکل ۱: مقایسه رفتار درونیابی لاگرانژ و اسپلاین مکعبی

تحلیل نتایج: همان‌طور که در نمودار مشخص است، چندجمله‌ای لاگرانژ برای گذر کردن از تمامی نقاط، در بازه انتهایی دچار نوسانات شدید شده است. در مقابل، اسپلاین مکعبی به دلیل استفاده از چندجمله‌ای‌های محلی، رفتاری کاملاً نرم و پایدار دارد. این خروجی ضعف کلاسیک درونیابی سراسری با چندجمله‌ای‌های درجه بالا را اثبات می‌کند.

بخش دوم: پردازش پویای داده‌ها و ترسیم نمودارهای آماری

ارزیابی زمان اجرای الگوریتم‌ها با حجم ورودی متغیر

۵ معرفی مسئله و بارگذاری پویا

در این بخش، هدف پردازش یک فایل اکسل شامل زمان اجرای ۳ الگوریتم مختلف به ازای داده‌هایی با اندازه ۱۰۰ الی ۷۰۰ کیلوبایت است. پیاده‌سازی به نحوی انجام شده که خواندن سطرها و ستون‌ها کاملاً به صورت خودکار بوده و با افزودن داده‌های جدید به فایل اکسل، نمودارها و تحلیل‌های آماری بدون نیاز به تغییر در کد منبع، به‌روزرسانی می‌شوند. داده‌های نهایی پردازش‌شده در جدول ۲ آورده شده‌اند.

جدول ۲: داده‌های زمان اجرای الگوریتم‌ها بر حسب میلی‌ثانیه

اندازه داده	الگوریتم ۱	الگوریتم ۲	الگوریتم ۳
۱۰۰KB	۱۲	۴۵	۹۰
۲۰۰KB	۲۵	۸۸	۱۹۰
۳۰۰KB	۳۳	۱۴۰	۳۱۰
۴۰۰KB	۴۸	۱۹۵	۴۲۰
۵۰۰KB	۶۰	۲۵۰	۵۵۰
۶۰۰KB	۷۲	۳۱۰	۶۸۰
۷۰۰KB	۸۰	۳۲۰	۷۰۰

۶ تحلیل آماری (بخش ج)

بر اساس خواسته‌های مسئله، میانگین زمان اجرای الگوریتم ۲ از فایل اکسل قرائت و در دو حالت مختلف محاسبه شد:

• میانگین در بازه اولیه (۱۰۰ تا ۶۰۰ کیلوبایت):

$$\mu_{100-600} = 250.00ms$$

• میانگین کل داده‌ها (سطر ۱۰۰ الی ۷۰۰ کیلوبایت):

$$\mu_{total} = 260.00ms$$

۷ کد پیاده‌سازی پایتون

کد زیر (توسعه‌یافته در فایل part2.py) وظیفه پاک‌سازی عمیق کاراکترهای پنهان، استخراج پارامترهای آماری و ترسیم هر سه نمودار را بر عهده دارد:

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # 1. Load the file dynamically (supports both real .xlsx and
renamed .csv)
5 file_path = 'data.xlsx'
6 try:
7     df = pd.read_excel(file_path)
8 except Exception:
9     df = pd.read_csv(file_path)
10
11 # 2. Deep clean headers
12 df.columns = df.columns.astype(str).str.strip().str.replace(' ',
'')
13
14 # 3. Set the very first column dynamically as X-axis index
15 first_col = df.columns[0]
16 df.set_index(first_col, inplace=True)
17 df.index = df.index.astype(str).str.strip().str.replace(' ', '')
18
19 # 4. Deep clean numeric values inside table cells
20 for col in df.columns:
21     if df[col].dtype == object:
22         df[col] = pd.to_numeric(df[col].astype(str).str.strip().str.replace
(' ', ''), errors='coerce')
23
24 # Line Plot
25 plt.figure(figsize=(8, 5), dpi=120)
26 for col in df.columns:
27     plt.plot(df.index, df[col], marker='o', linewidth=2, label=col)
28 plt.title('Execution Time vs Data Size (Line Plot)')
29 plt.xlabel('Data Size')
30 plt.ylabel('Execution Time')
31 plt.legend()
32 plt.grid(True, linestyle='--')
33 plt.savefig('part2_line_plot.png', dpi=300)
34 plt.close()
35
36 # Bar Plot
37 ax = df.plot(kind='bar', figsize=(8, 5), width=0.7, zorder=3)
```

```

38     ax.set_title('Execution Time by Algorithm (Bar Plot)')
39     ax.set_xlabel('Data Size')
40     ax.set_ylabel('Execution Time')
41     ax.grid(axis='y', linestyle='--', zorder=0)
42     plt.xticks(rotation=0)
43     plt.tight_layout()
44     plt.savefig('part2_bar_plot.png', dpi=300)
45     plt.close()
46
47     # Box Plot
48     plt.figure(figsize=(7, 5), dpi=120)
49     df.boxplot(column=list(df.columns), grid=True)
50     plt.title('Execution Time Distribution (Box Plot)')
51     plt.ylabel('Execution Time')
52     plt.savefig('part2_box_plot.png', dpi=300)
53     plt.close()
54
55     # Statistical Calculations for Alg.2
56     alg2_col = df.columns[1]
57     target_sizes = ['100KB', '200KB', '300KB', '400KB', '500KB', '600KB
']
58     subset_mean = df.loc[df.index.isin(target_sizes), alg2_col].mean()
59     total_mean = df[alg2_col].mean()
60     print("SUCCESS: All 3 plots generated successfully!")
61     print(f"Mean execution time (100KB - 600KB) : {subset_mean:.2f}")
62     print(f"Overall Mean execution time (All rows): {total_mean:.2f}")
63

```

۸ خروجی اجرای برنامه در ترمینال

خروجی متنی ثبت شده در کنسول پس از پردازش موفقیت آمیز فایل اکسل به شرح زیر است:

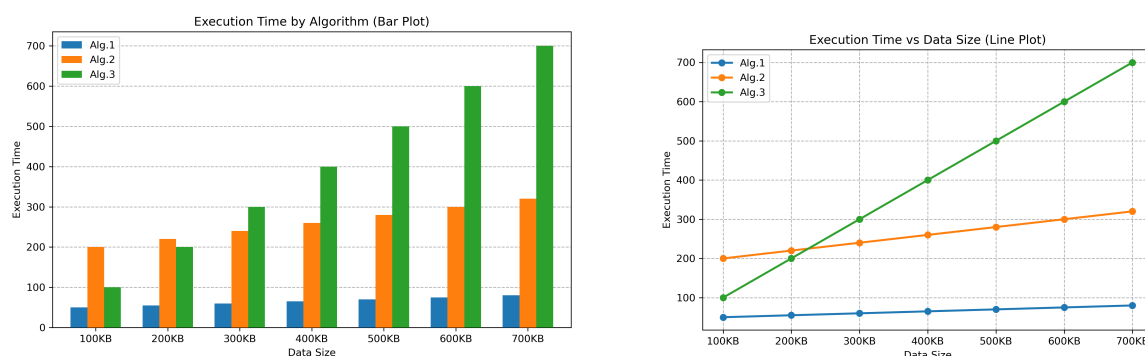
```

1     SUCCESS: All 3 plots generated successfully!
2
3     === Statistical Analysis for [Alg.2] ===
4     Mean execution time (100KB - 600KB) : 250.00
5     Overall Mean execution time (All rows): 260.00
6

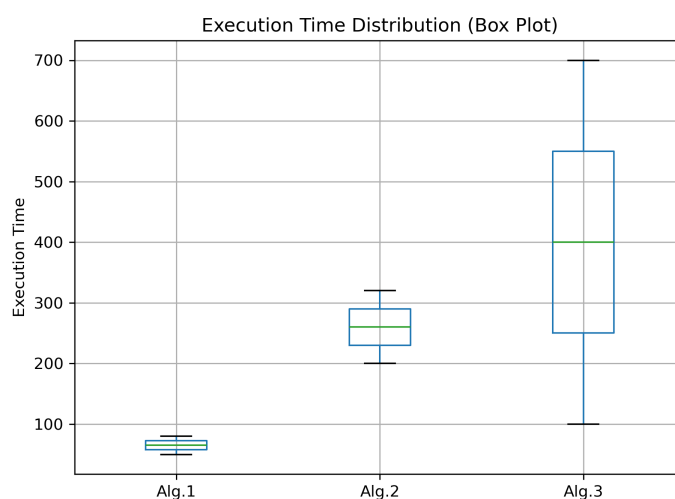
```

۹ خروجی‌های بصری و تحلیل عملکرد

سه نمودار ترسیم‌شده توسط نرم‌افزار در شکل‌های ۲ و ۳ نمایش داده شده‌اند.



شکل ۲: نمودار خطی روند رشد زمان اجرا (راست) و نمودار میله‌ای مقایسه‌ای (چپ)



شکل ۳: نمودار جعبه‌ای (Box Plot) جهت نمایش توزیع و پراکندگی زمان اجرا

تحلیل فنی نتایج: با بررسی نمودار خطی و میله‌ای مشخص می‌شود که **الگوریتم ۱** دارای ملایم‌ترین شیب رشد زمانی بوده و برای پردازش داده‌های مقیاس بزرگ، بهینه‌ترین انتخاب است (پیچیدگی زمانی شبه‌خطی). در سمت مقابل، **الگوریتم ۳** با افزایش حجم ورودی از ۱۰۰ به ۷۰۰ کیلوبایت دچار جهش شدید زمانی می‌شود.

همچنین در نمودار جعبه‌ای (شکل ۳)، کشیدگی دامنه بین چارک اول تا سوم برای الگوریتم ۳، نشان‌دهنده واریانس و بی‌ثباتی شدید زمان اجرای این الگوریتم نسبت به الگوریتم ۱ و ۲ است.

بخش سوم: انتگرال‌گیری عددی توابع نمایی

مقایسه دقت روش‌های دوزنقه‌ای، سیمپسون و تربیع گوسی

۱۰ معرفی مسئله و اهمیت روش‌های عددی

در این بخش، هدف محاسبه انتگرال معین تابع نمایی $f(x) = e^{x^2}$ در بازه استاندارد $[0, 1]$ است. از نظر تحلیلی، این انتگرال دارای تابع اولیه بر حسب توابع جبری مقدماتی نبوده و پاسخ دقیق آن صرفاً با استفاده از «تابع خطا» (Error Function) قابل بیان است؛ از این رو استفاده از روش‌های انتگرال‌گیری عددی برای حل آن الزامی است.

در این پیاده‌سازی، تقریب انتگرال با سه متد **دوزنقه‌ای** (Trapezoidal)، **سیمپسون** (Simpson) و **تربیع گوسی** (Gaussian Quadrature) انجام شده و نتایج مورد ارزیابی قرار گرفته‌اند.

۱۱ کد پیاده‌سازی پایتون

کد زیر توسعه‌یافته در فایل `part3.py`، محاسبات عددی را روی یک شبکه گسسته با ۱۰۰۱ نقطه پیاده‌سازی کرده و خطاها را نسبت به متد گوسی بسط می‌دهد:

```
1 import numpy as np
2 from scipy.integrate import trapezoid, simpson, quad
3 import matplotlib.pyplot as plt
4
5 # 1. Define the target mathematical function: f(x) = e^(x^2)
6 def f(x):
7     return np.exp(x**2)
8
9 # Define integration limits [a, b]
10 a, b = 0, 1
11
12 # Generate an evenly spaced grid of N points (Odd number required
for Simpson)
13 N = 1001
14 x_grid = np.linspace(a, b, N)
15 y_grid = f(x_grid)
16
17 # Part A: Trapezoidal and Simpson's Rule
18 trap_result = trapezoid(y_grid, x_grid)
19 simp_result = simpson(y_grid, x=x_grid)
20
21 # Part B: Gaussian Quadrature (High Precision Reference)
22 gauss_result, estimated_error = quad(f, a, b)
```



```

23     print("=== Numerical Integration Results for f(x) = e^(x^2) over
24     [0, 1] ===")
25     print(f"1. Trapezoidal Rule   : {trap_result:.10f}")
26     print(f"2. Simpson's Rule    : {simp_result:.10f}")
27     print(f"3. Gaussian Quad     : {gauss_result:.10f}")
28

```

۱۲ خروجی اجرای برنامه در کنسول

خروجی متنی ثبت شده در ترمینال پایتون که حاوی مقادیر انتگرال تا ۱۰ رقم اعشار و تحلیل خطا است، به شرح زیر است:

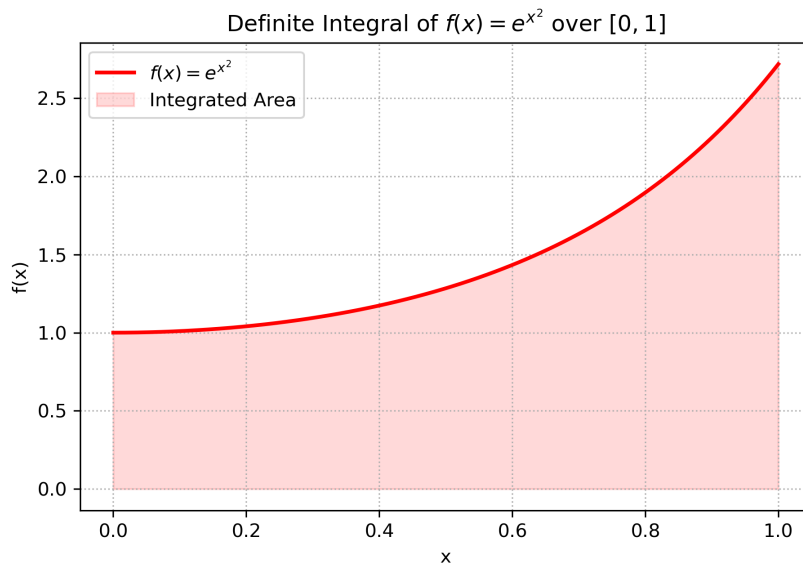
```

1     === Numerical Integration Results for f(x) = e^(x^2) over [0,
2     1] ===
3     1. Trapezoidal Rule   : 1.4626521990
4     2. Simpson's Rule    : 1.4626517459
5     3. Gaussian Quad     : 1.4626517459 (Absolute Error Margin:
6     1.62e-14)
7
8     === Error Analysis (Compared to Gaussian Quadrature) ===
9     Trapezoid Absolute Error : 4.53e-07
10    Simpson Absolute Error   : 3.02e-13

```

۱۳ خروجی بصری و مقایسه تحلیلی نتایج

نمودار تابع $f(x) = e^{x^2}$ و سطح محصور زیر منحنی در شکل ۴ نمایش داده شده است.



شکل ۴: نمودار تابع نمایی $f(x) = e^{x^2}$ و مساحت تحت انتگرال در بازه $[0, 1]$

تحلیل مهندسی و ریاضی نتایج: با مبنا قرار دادن روش **تربیع گوسی** به عنوان دقیق‌ترین پاسخ مرجع (1.4626517459)، مشاهده می‌شود که روش **سیمپسون** با خطای مطلق در حد 3.02×10^{-13} عملکردی فوق‌العاده دقیق داشته و تا ۹ رقم اعشار با پاسخ واقعی یکسان است. در مقابل، روش **دوزنقه‌ای** از رقم اعشار ششم دچار اختلاف شده و خطایی برابر با 4.53×10^{-7} ثبت کرده است. دلیل برتری مطلق متد سیمپسون در این مسئله، ماهیت هندسی تابع e^{x^2} است؛ این تابع دارای انحنای رو به بالای (Convexity) شدیدی است. روش دوزنقه‌ای با تقریب زدن این انحنای توسط خطوط راست، مساحت کلاهای بالای بازه‌ها را از دست داده و دچار خطای ساختاری می‌شود، اما روش سیمپسون با تقریب زدن تابع توسط سهمی‌های درجه ۲، کاملاً بر انحنای تابع منطبق می‌گردد.

۱۴ مخزن عمومی کدها (GitHub Repository)

بر اساس الزامات پروژه، تمامی کدهای توسعه‌یافته برای بخش ۱، ۲ و ۳ به همراه فایل‌های گزارش و نمودارها، در یک مخزن عمومی در گیت‌هاب با سابقه کامیت‌های مجزا آپلود شده‌اند. مخزن مذکور از طریق آدرس زیر در دسترس است:

<https://github.com/itsaboutarman/Project03>